

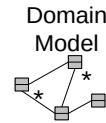


Software Design

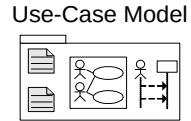


Sample UP Artifact Relationships

Business Modeling

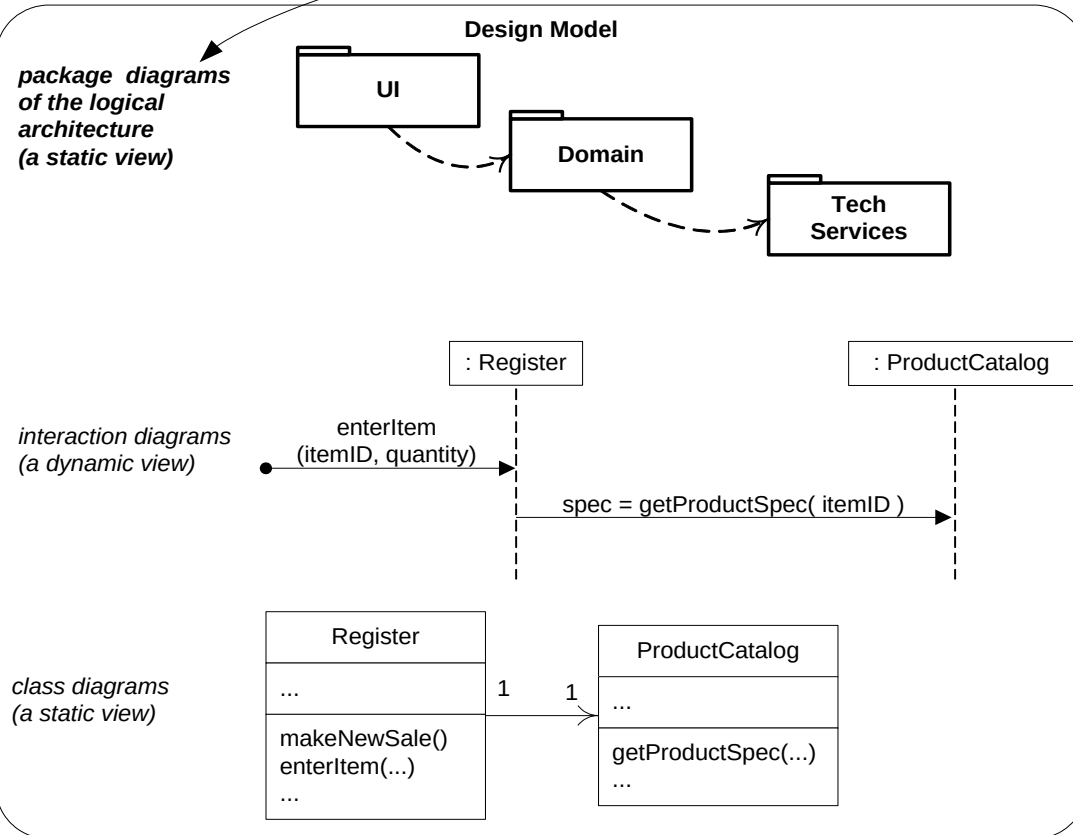


Requirements



The logical architecture is influenced by the constraints and non-functional requirements captured in the Supp. Spec.

Design



Different Design Aspects

- ▶ *Architecture design:*
 - ▶ The division into subsystems and components,
 - How these will be connected.
 - How they will interact.
 - Their interfaces.
- ▶ *Class design:*
 - ▶ The various features of classes.
- ▶ *User interface design*
- ▶ *Algorithm design:*
 - ▶ The design of computational mechanisms.
- ▶ *Protocol design:*
 - ▶ The design of communications protocol.



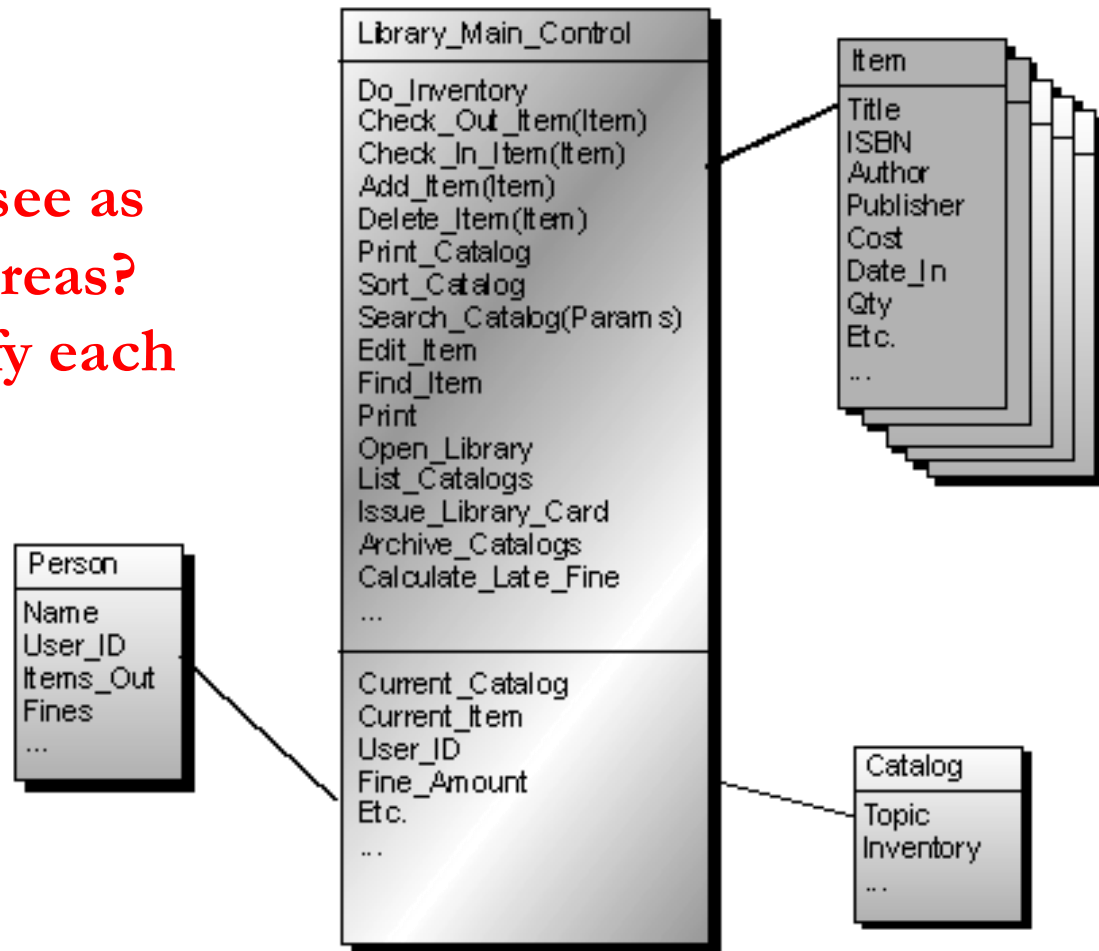
What is Software Design?

- ▶ A software design expresses a solution to a problem in programming language independent terms.
- ▶ This permits a design to be implemented in any programming language.



Simple Library system - Existing design

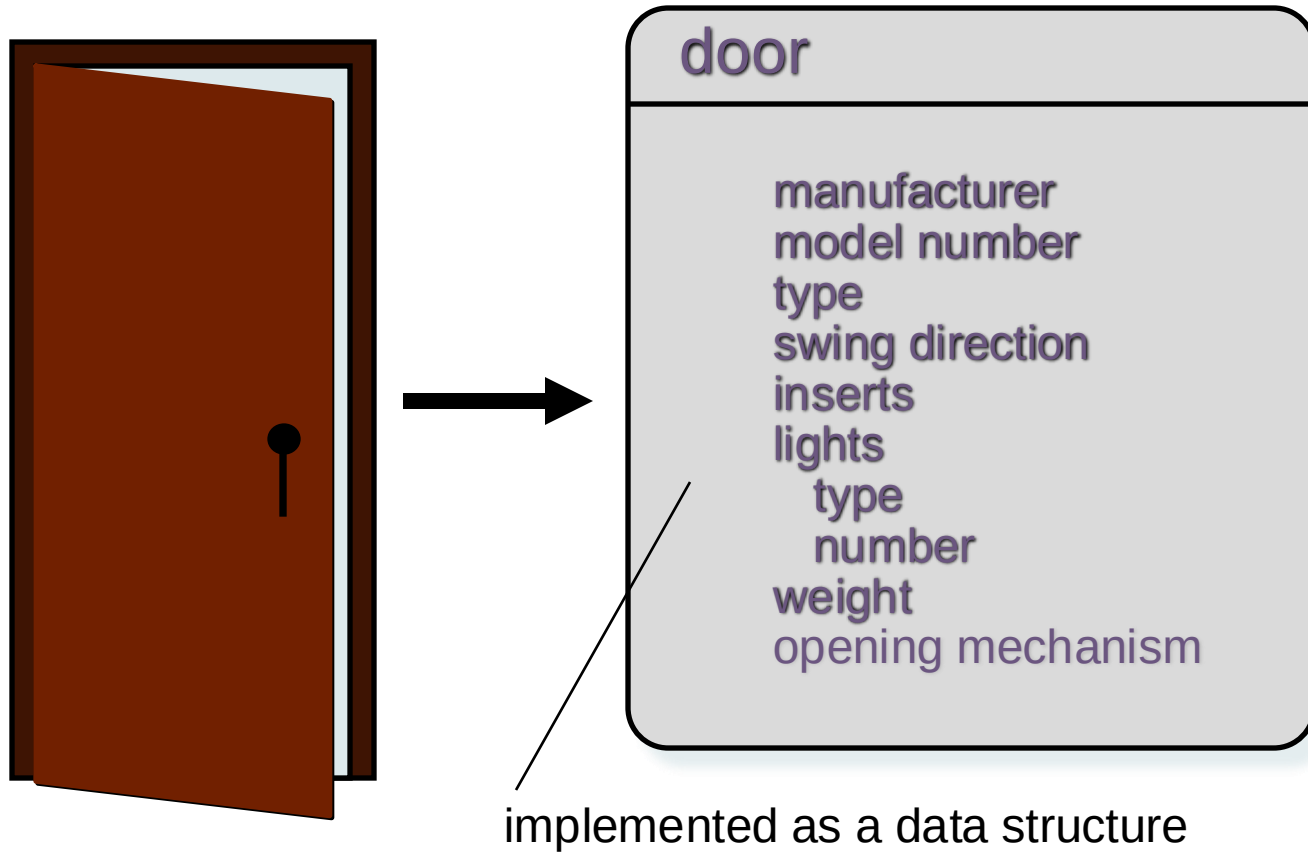
What areas do you see as potential problem areas?
Why did you identify each of those areas?



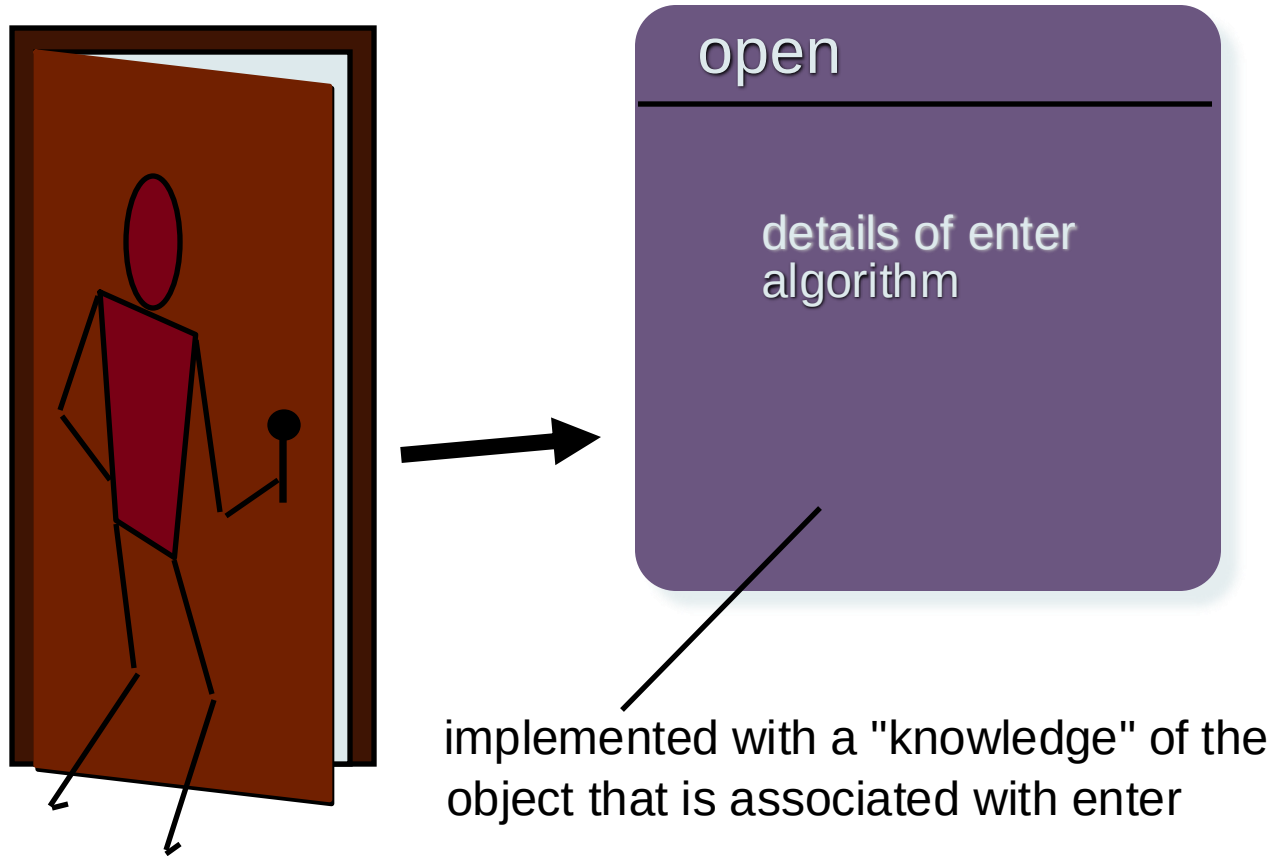
Fundamental Concepts

- ▶ **Abstraction**—data, procedure, control
- ▶ **Separation of concerns**—any complex problem can be more easily handled if it is subdivided into pieces
- ▶ **Modularity**—compartmentalization of data and function
- ▶ **Hiding**—controlled interfaces
- ▶ **Refinement**—elaboration of detail for all abstractions
- ▶ **Design Classes**—provide design detail that will enable analysis classes to be implemented
- ▶ **Functional independence**—High Cohesion and Low coupling
- ▶ **Patterns**—”conveys the essence” of a proven design solution

Data Abstraction



Procedural Abstraction



Separation of Concerns

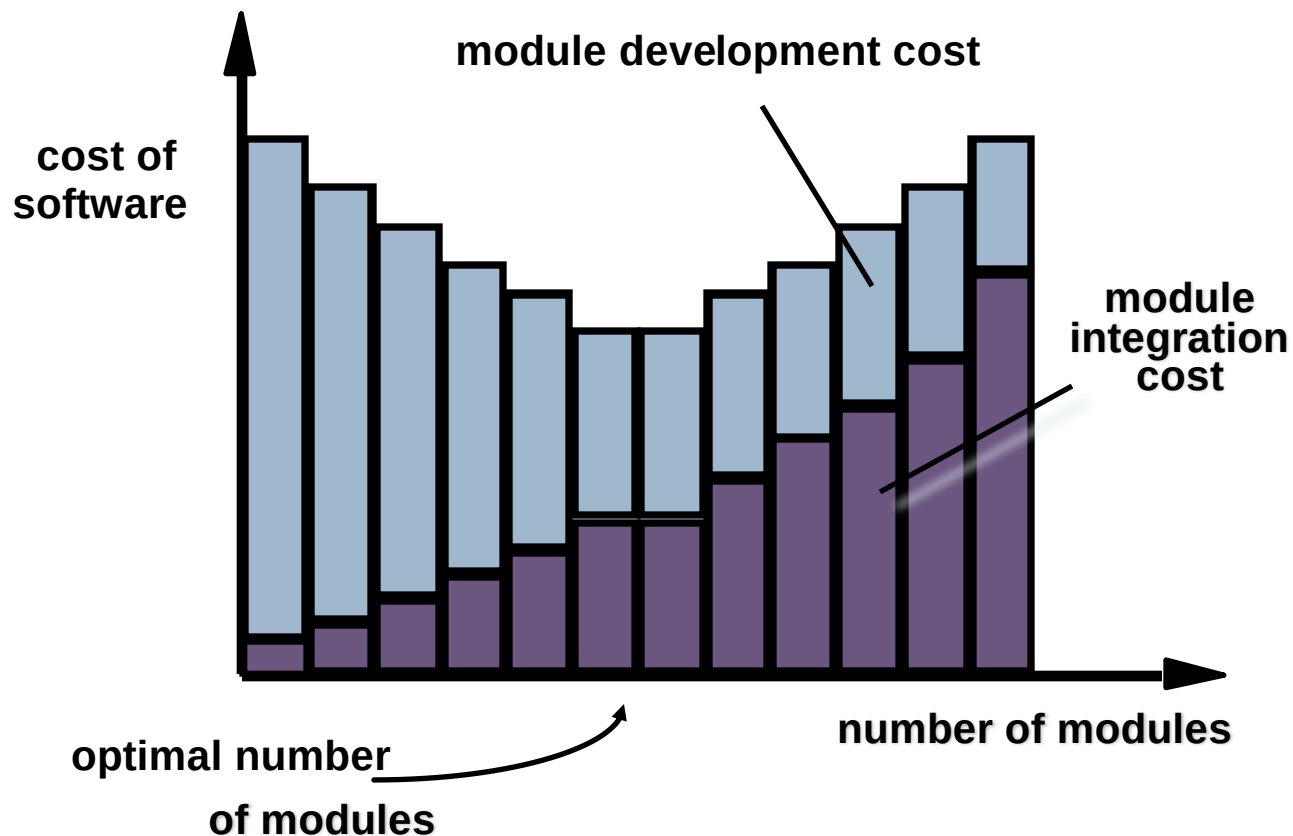
- ▶ Any complex problem can be more easily handled if it is **subdivided into pieces** that can each be solved and/or optimized independently
- ▶ A *concern* is a feature or behavior that is specified as part of the requirements model for the software
- ▶ By separating concerns into smaller, and therefore more manageable pieces, a problem takes less effort and time to solve.

Modularity

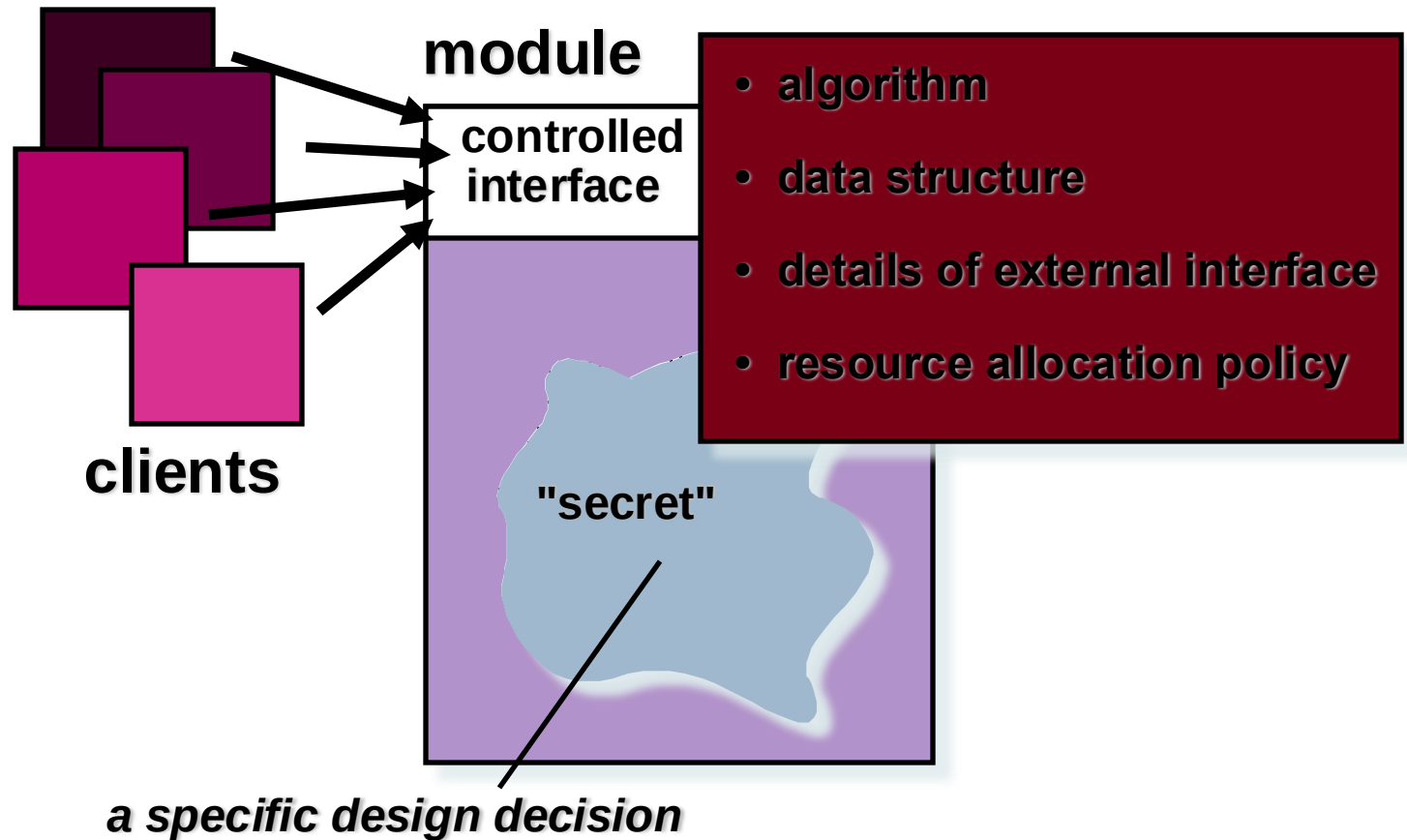
- ▶ "modularity is the single attribute of software that allows a program to be intellectually manageable" [Mye78].
- ▶ **Monolithic software (i.e., a large program composed of a single module) cannot be easily grasped by a software engineer.**
- ▶ In almost all instances, you should break the design into many modules, hoping to make understanding easier and as a consequence, reduce the cost required to build the software.

Modularity: Trade-offs

What is the "right" number of modules for a specific software design?



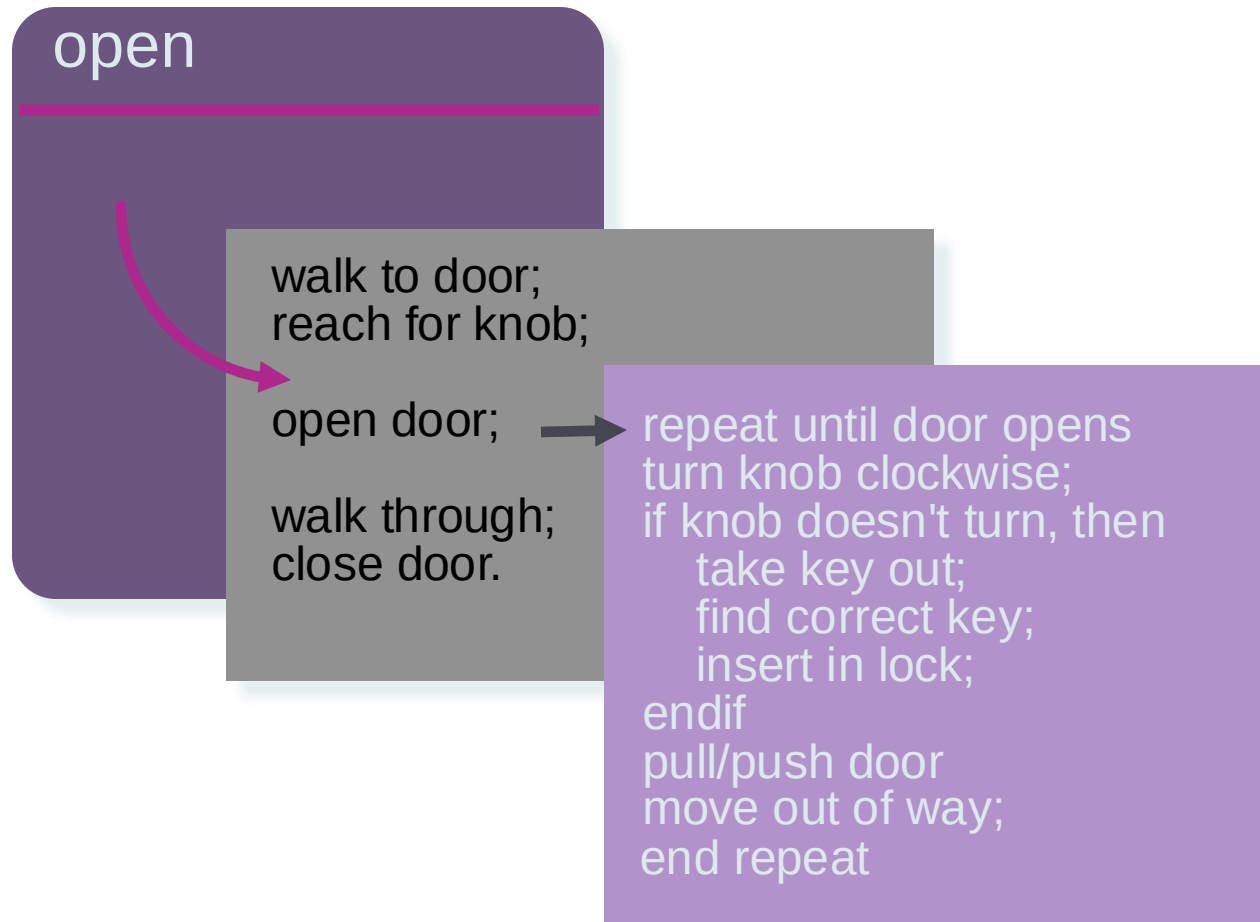
Information Hiding



Why Information Hiding?

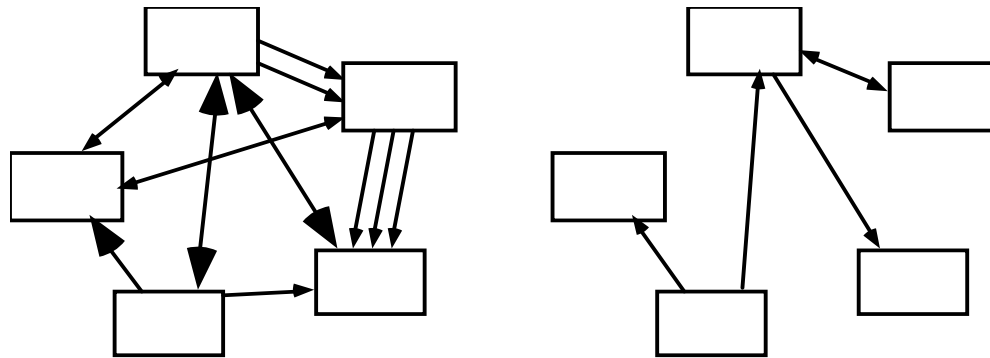
- ▶ reduces the likelihood of “side effects”
- ▶ limits the global impact of **local design** decisions
- ▶ emphasizes communication through **controlled interfaces**
- ▶ discourages the use of global data
- ▶ leads to **encapsulation**—an attribute of high quality design
- ▶ results in higher quality software

Stepwise Refinement



Coupling

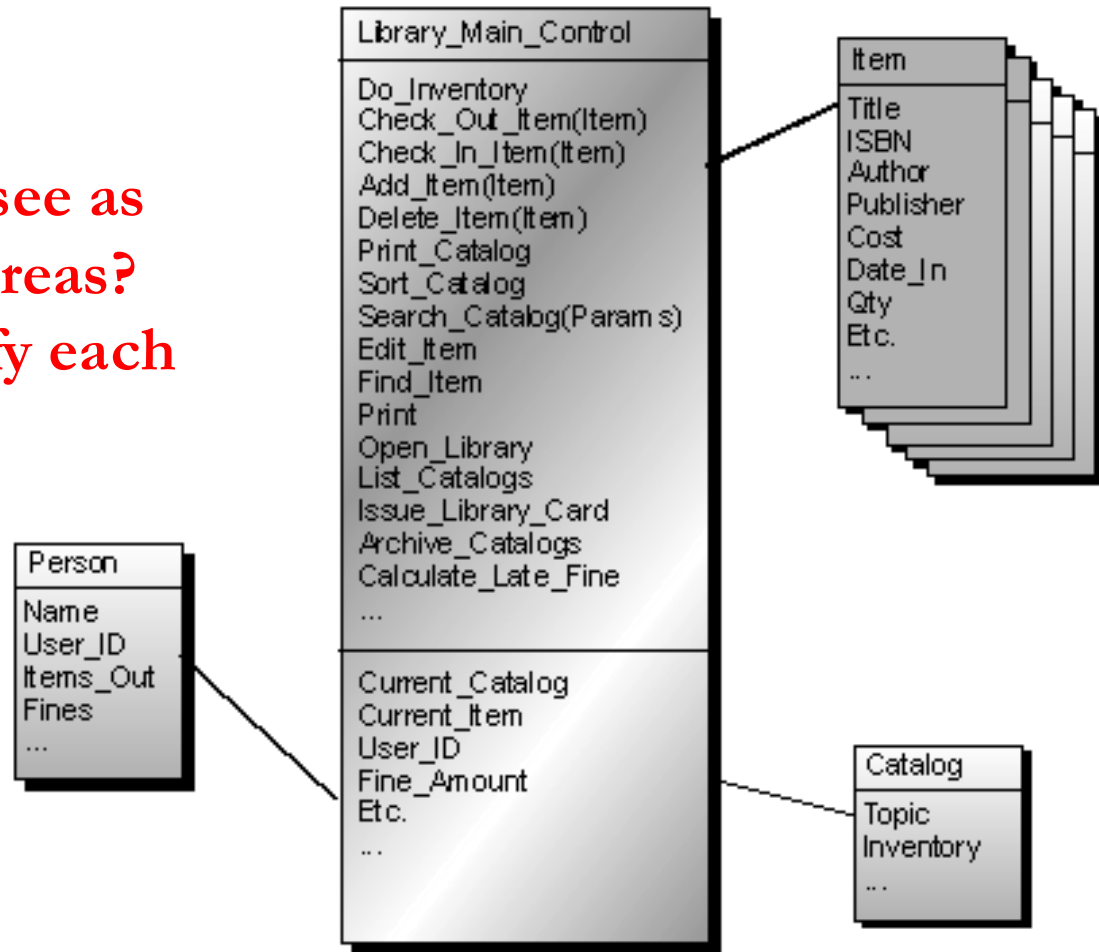
- ▶ *Coupling* occurs when there are *interdependencies* between one module and another



- ▶ When interdependencies exist, changes in one place will require changes somewhere else.
- ▶ A network of interdependencies makes it hard to see at a glance how some component works.

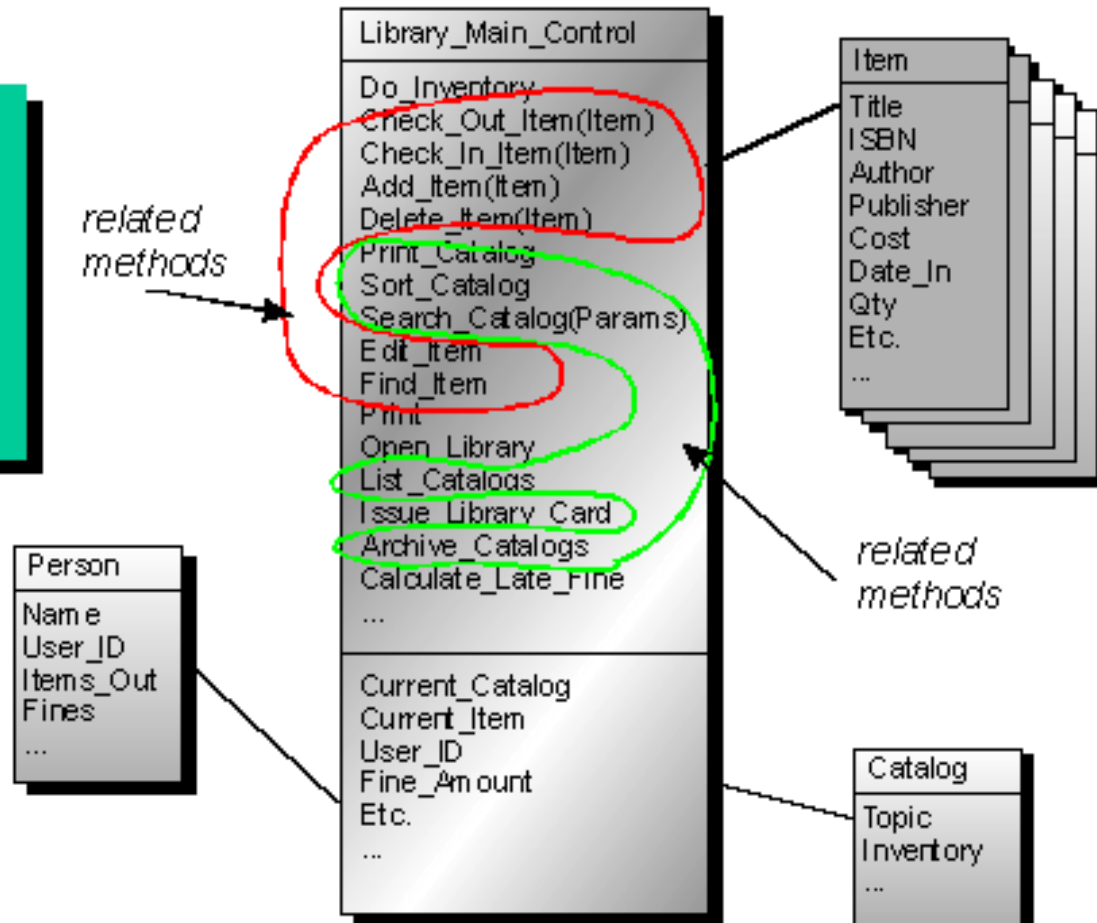
Revisting Library system - Existing design

What areas do you see as potential problem areas?
Why did you identify each of those areas?



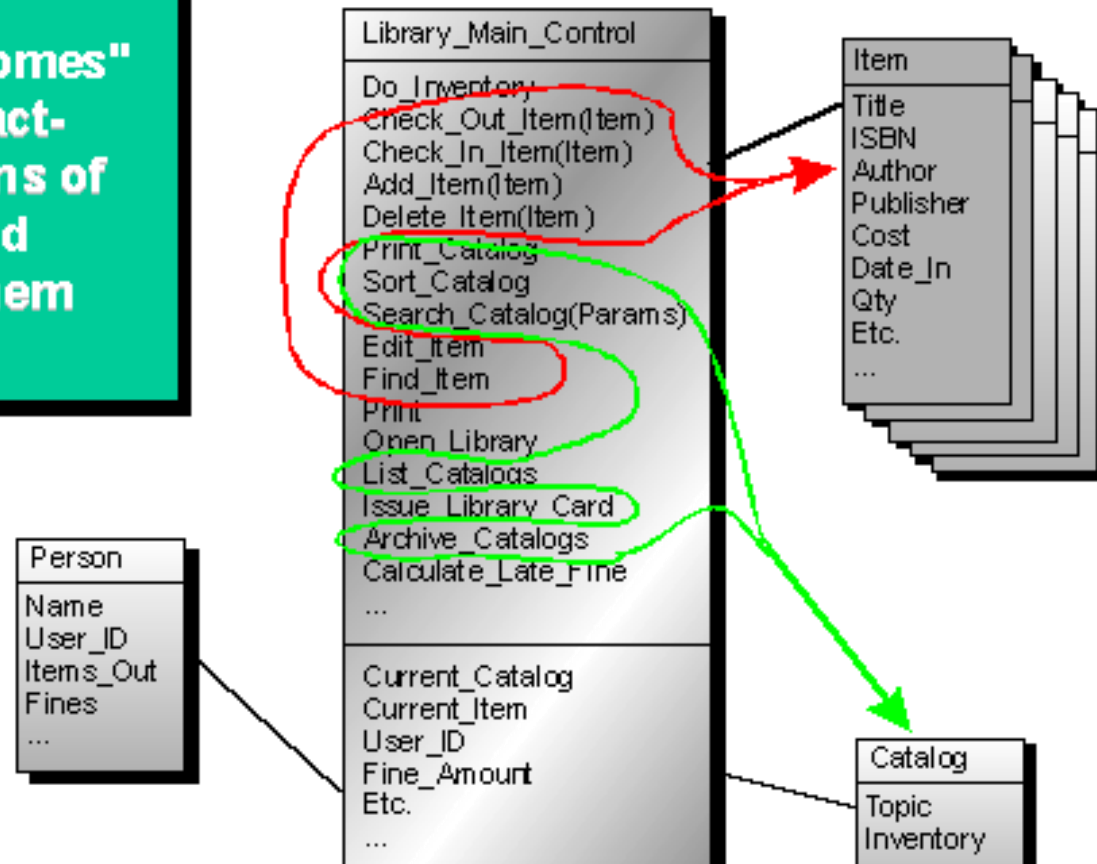
Library system – Changing the design

Step 1:
Identify or categorize
related attributes and
operations according
to contracts.



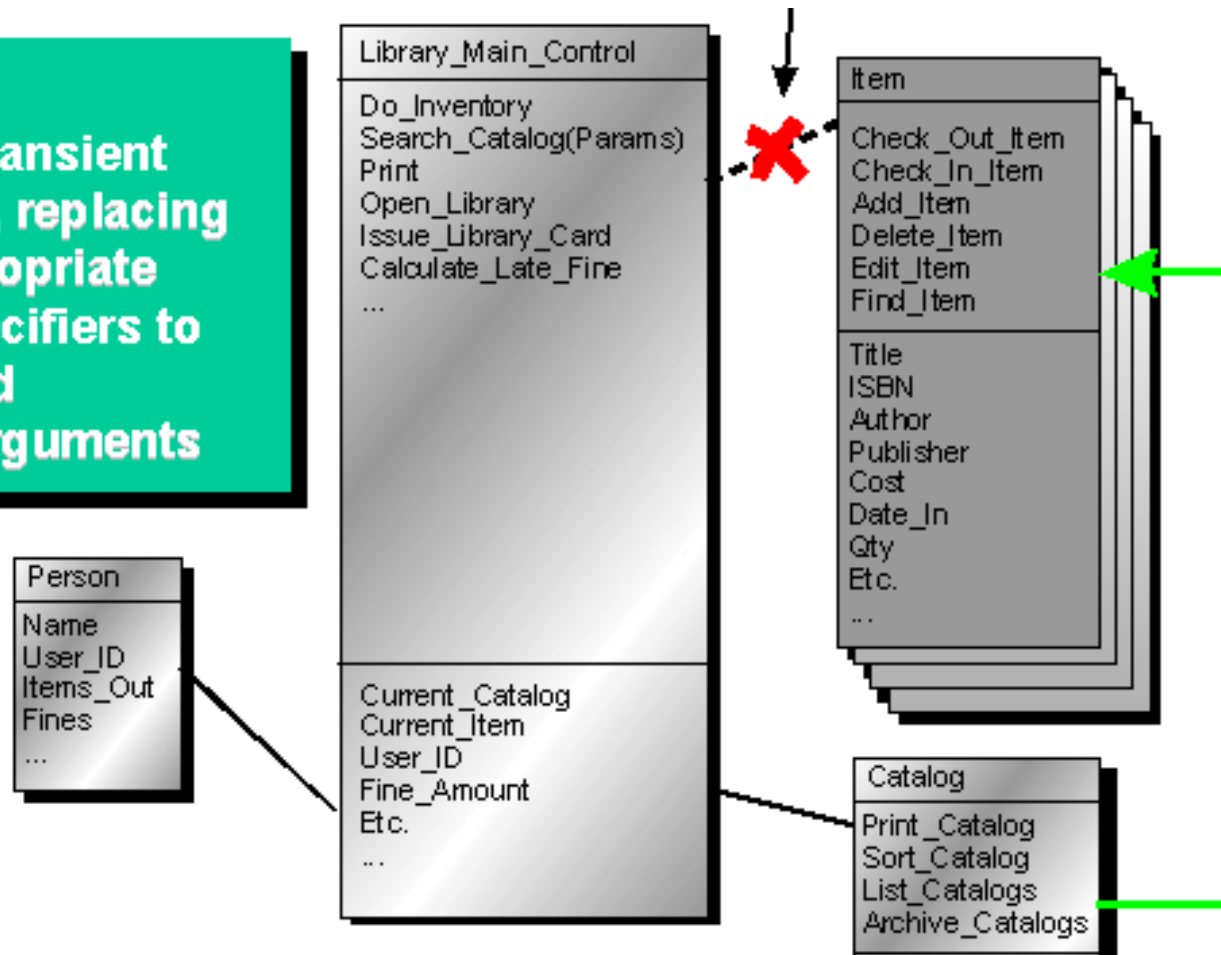
Library system – Changing the design

Step 2:
Find "natural homes"
for these contract-
based collections of
functionality and
them migrate them
there



Library system – Changing the design

**Final Step:
Remove all transient
associations, replacing
them as appropriate
with type specifiers to
attributes and
operations arguments**



Library system – Changing the design

